

USAspending Gold Layer — T-SQL Reference

Fabric Data Warehouse implementation: star schema, incremental refresh, and Row-Level Security

Overview

This document contains the complete T-SQL implementation of the Gold layer for the USAspending portfolio project, built on a Fabric Data Warehouse (as opposed to a Lakehouse/Spark notebook). It covers: the star schema (dimension + fact tables), a watermark-based incremental refresh pattern using MERGE statements, native Row-Level Security enforced at the database tier via CREATE SECURITY POLICY, and the stored procedures needed to orchestrate the whole load.

Important caveat (from Microsoft's own documentation): this Row-Level Security only applies to queries that go through the Warehouse's SQL endpoint. If a user can access the same underlying data directly through Spark or OneLake, they bypass this RLS entirely. This is a database-tier security control, not a storage-tier one.

1. Schema and Tables

1.1 Schema for RLS objects

```
CREATE SCHEMA rls;  
GO
```

1.2 Dimension tables

Each dimension reserves surrogate key -1 for an explicit 'Unknown' member (see Section 1.6). This is populated by the seed statements in that section, not by the CREATE TABLE itself.

```
CREATE TABLE dbo.dim_agency (  
    agency_key INT IDENTITY(1,1) NOT NULL,  
    agency_code VARCHAR(10) NOT NULL,  
    awarding_agency_name VARCHAR(200),  
    funding_agency_name VARCHAR(200)  
);  
  
CREATE TABLE dbo.dim_account (  
    account_key INT IDENTITY(1,1) NOT NULL,  
    federal_account_symbol VARCHAR(20) NOT NULL,  
    federal_account_name VARCHAR(300),  
    budget_function VARCHAR(200),  
    budget_subfunction VARCHAR(200),  
    object_class_code VARCHAR(10),  
    object_class_name VARCHAR(200)  
);  
  
CREATE TABLE dbo.dim_program_activity (  
    program_activity_key INT IDENTITY(1,1) NOT NULL,  
    program_activity_code VARCHAR(20) NOT NULL,  
    program_activity_name VARCHAR(300)  
);  
  
CREATE TABLE dbo.dim_recipient (  
    recipient_key INT IDENTITY(1,1) NOT NULL,  
    recipient_uei VARCHAR(20) NOT NULL,  
    recipient_name VARCHAR(300),  
    recipient_state VARCHAR(50),
```

```

        recipient_country VARCHAR(50),
        recipient_city VARCHAR(100)
    );

```

1.3 Fact table

fiscal_year and period_number are derived from submission_period (e.g. 'FY2025P02') and stored physically on the fact table, matching the notebook/Direct Lake implementation — Direct Lake does not support DAX calculated columns, so derived fields must be computed upstream, in the layer that writes the table, not in the semantic model.

```

CREATE TABLE dbo.fact_award (
    agency_key INT,
    account_key INT,
    program_activity_key INT,
    recipient_key INT,
    award_unique_key VARCHAR(200) NOT NULL,
    parent_award_id_piid VARCHAR(50),
    federal_account_symbol VARCHAR(20),
    program_activity_code VARCHAR(20),
    program_activity_name VARCHAR(300),
    object_class_code VARCHAR(10),
    direct_or_reimbursable_funding_source VARCHAR(5),
    disaster_emergency_fund_code VARCHAR(5),
    submission_period VARCHAR(10),
    program_activity_reporting_key VARCHAR(50),
    AwardCategory VARCHAR(20) NOT NULL,
    transaction_obligated_amount DECIMAL(18,2),
    gross_outlay_amount_FYB_to_period_end DECIMAL(18,2),
    last_modified_date DATE,
    agency_code VARCHAR(10) NOT NULL,
    fiscal_year VARCHAR(6),
    period_number TINYINT
);

```

1.4 RLS mapping table

```

CREATE TABLE dbo.dim_security_mapping (
    user_email VARCHAR(200) NOT NULL,
    agency_code VARCHAR(10) NOT NULL
);

INSERT INTO dbo.dim_security_mapping VALUES
    ('sba.analyst@agency.gov', 'SBA'),
    ('epa.analyst@agency.gov', 'EPA'),
    ('dol.analyst@agency.gov', 'DOL'),
    ('va.analyst@agency.gov', 'VA'),
    ('admin@agency.gov', 'ALL');

```

1.5 Watermark control table

```

CREATE TABLE dbo.gold_watermark (
    table_name VARCHAR(100) NOT NULL,
    last_watermark DATETIME2 NOT NULL
);

INSERT INTO dbo.gold_watermark VALUES
    ('dim_agency', '1900-01-01'),
    ('dim_account', '1900-01-01'),
    ('dim_program_activity', '1900-01-01'),
    ('dim_recipient', '1900-01-01'),

```

```
('fact_award', '1900-01-01');
```

1.6 'Unknown' member seed rows

Reserves surrogate key -1 in every dimension, so a fact row whose business key genuinely has no match (confirmed real in this project -- e.g. NULL program_activity_code on ~358,600 rows -- not a join bug) resolves to a visible 'Unknown' member instead of a raw NULL foreign key, which can cause a row to silently disappear from a Direct Lake visual sliced by that dimension.

```
INSERT INTO dbo.dim_agency (agency_key, agency_code, awarding_agency_name, funding_agency_name)
SELECT -1, 'UNKNOWN', 'Unknown', 'Unknown'
WHERE NOT EXISTS (SELECT 1 FROM dbo.dim_agency WHERE agency_key = -1);

INSERT INTO dbo.dim_account (account_key, federal_account_symbol, federal_account_name,
    budget_function, budget_subfunction, object_class_code, object_class_name)
SELECT -1, 'UNKNOWN', 'Unknown', 'Unknown', 'Unknown', 'UNKNOWN', 'Unknown'
WHERE NOT EXISTS (SELECT 1 FROM dbo.dim_account WHERE account_key = -1);

INSERT INTO dbo.dim_program_activity (program_activity_key, program_activity_code, program_activity_name)
SELECT -1, 'UNKNOWN', 'Unknown'
WHERE NOT EXISTS (SELECT 1 FROM dbo.dim_program_activity WHERE program_activity_key = -1);

INSERT INTO dbo.dim_recipient (recipient_key, recipient_uei, recipient_name,
    recipient_state, recipient_country, recipient_city)
SELECT -1, 'UNKNOWN', 'Unknown', 'Unknown', 'Unknown', 'Unknown'
WHERE NOT EXISTS (SELECT 1 FROM dbo.dim_recipient WHERE recipient_key = -1);
```

2. Row-Level Security

USER_NAME() resolves to the connected Microsoft Entra ID identity's UPN when users authenticate via Entra auth. This makes the filter dynamic and tied to whoever is actually logged in at query time — not a static, pre-tested value.

```
CREATE FUNCTION rls.fn_agency_predicate(@agency_code VARCHAR(10))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS is_visible
    WHERE EXISTS (
        SELECT 1 FROM dbo.dim_security_mapping sm
        WHERE sm.user_email = USER_NAME()
        AND (sm.agency_code = @agency_code OR sm.agency_code = 'ALL')
    );
GO

CREATE SECURITY POLICY rls.AgencyFilter
ADD FILTER PREDICATE rls.fn_agency_predicate(agency_code) ON dbo.fact_award,
ADD FILTER PREDICATE rls.fn_agency_predicate(agency_code) ON dbo.dim_agency
WITH (STATE = ON);
```

3. Cross-Item Read from Silver (Lakehouse)

A Fabric Warehouse can query a Lakehouse's tables directly via three-part naming, since both items share one workspace metastore.

```
CREATE VIEW dbo.vw_silver_unioned AS
SELECT *, 'assistance' AS AwardCategory FROM lh_silver_usaspending.dbo.silver_assistance
UNION ALL
SELECT *, 'contracts' AS AwardCategory FROM lh_silver_usaspending.dbo.silver_contracts;
```

Note: AwardCategory retains its original casing here as a derived label column (not a source field), distinct from the lowercase snake_case used for actual data columns throughout this schema.

4. Incremental Load Procedures — Dimensions

Each dimension procedure follows the same shape: read the stored watermark, MERGE new/changed business-key rows from Silver, then advance the watermark to the newest row processed.

4.1 usp_LoadDimAgency

```
CREATE PROCEDURE dbo.usp_LoadDimAgency
AS
BEGIN
    DECLARE @watermark DATETIME2 = (SELECT last_watermark FROM dbo.gold_watermark WHERE table_name = 'dim_agency');
    DECLARE @max_ts DATETIME2;

    MERGE INTO dbo.dim_agency AS t
    USING (
        SELECT DISTINCT agency_code, awarding_agency_name, funding_agency_name
        FROM dbo.vw_silver_unioned
        WHERE _silver_load_ts > @watermark
    ) AS s
    ON t.agency_code = s.agency_code
    WHEN MATCHED THEN UPDATE SET
        t.awarding_agency_name = s.awarding_agency_name,
        t.funding_agency_name = s.funding_agency_name
    WHEN NOT MATCHED THEN INSERT (agency_code, awarding_agency_name, funding_agency_name)
        VALUES (s.agency_code, s.awarding_agency_name, s.funding_agency_name);

    SELECT @max_ts = MAX(_silver_load_ts) FROM dbo.vw_silver_unioned;
    IF @max_ts IS NOT NULL
        UPDATE dbo.gold_watermark SET last_watermark = @max_ts WHERE table_name = 'dim_agency';
END
GO
```

4.2 usp_LoadDimAccount

```
CREATE PROCEDURE dbo.usp_LoadDimAccount
AS
BEGIN
    DECLARE @watermark DATETIME2 = (SELECT last_watermark FROM dbo.gold_watermark WHERE table_name = 'dim_account');
    DECLARE @max_ts DATETIME2;

    MERGE INTO dbo.dim_account AS t
    USING (
        SELECT DISTINCT federal_account_symbol, federal_account_name, budget_function,
            budget_subfunction, object_class_code, object_class_name
        FROM dbo.vw_silver_unioned
        WHERE _silver_load_ts > @watermark
    ) AS s
    ON t.federal_account_symbol = s.federal_account_symbol
    WHEN MATCHED THEN UPDATE SET
        t.federal_account_name = s.federal_account_name,
        t.budget_function = s.budget_function,
        t.budget_subfunction = s.budget_subfunction,
        t.object_class_code = s.object_class_code,
        t.object_class_name = s.object_class_name
    WHEN NOT MATCHED THEN INSERT (
        federal_account_symbol, federal_account_name, budget_function,
        budget_subfunction, object_class_code, object_class_name
    ) VALUES (
        s.federal_account_symbol, s.federal_account_name, s.budget_function,
        s.budget_subfunction, s.object_class_code, s.object_class_name
    );
END
```

```

SELECT @max_ts = MAX(_silver_load_ts) FROM dbo.vw_silver_unioned;
IF @max_ts IS NOT NULL
    UPDATE dbo.gold_watermark SET last_watermark = @max_ts WHERE table_name = 'dim_account';
END
GO

```

4.3 usp_LoadDimProgramActivity

```

CREATE PROCEDURE dbo.usp_LoadDimProgramActivity
AS
BEGIN
    DECLARE @watermark DATETIME2 = (SELECT last_watermark FROM dbo.gold_watermark WHERE table_name = 'dim_program_activity');
    DECLARE @max_ts DATETIME2;

    MERGE INTO dbo.dim_program_activity AS t
    USING (
        SELECT DISTINCT program_activity_code, program_activity_name
        FROM dbo.vw_silver_unioned
        WHERE _silver_load_ts > @watermark
    ) AS s
    ON t.program_activity_code = s.program_activity_code
    AND t.program_activity_name = s.program_activity_name
    WHEN NOT MATCHED THEN INSERT (program_activity_code, program_activity_name)
    VALUES (s.program_activity_code, s.program_activity_name);

    SELECT @max_ts = MAX(_silver_load_ts) FROM dbo.vw_silver_unioned;
    IF @max_ts IS NOT NULL
        UPDATE dbo.gold_watermark SET last_watermark = @max_ts WHERE table_name = 'dim_program_activity';
END
GO

```

Note: this dimension is matched on BOTH program_activity_code and program_activity_name together, since earlier investigation on this project confirmed the same code can legitimately pair with different names (see Section 6 note).

4.4 usp_LoadDimRecipient

```

CREATE PROCEDURE dbo.usp_LoadDimRecipient
AS
BEGIN
    DECLARE @watermark DATETIME2 = (SELECT last_watermark FROM dbo.gold_watermark WHERE table_name = 'dim_recipient');
    DECLARE @max_ts DATETIME2;

    MERGE INTO dbo.dim_recipient AS t
    USING (
        SELECT DISTINCT recipient_uei, recipient_name, recipient_state, recipient_country, recipient_city
        FROM dbo.vw_silver_unioned
        WHERE _silver_load_ts > @watermark
        AND recipient_uei IS NOT NULL
    ) AS s
    ON t.recipient_uei = s.recipient_uei
    WHEN MATCHED THEN UPDATE SET
        t.recipient_name = s.recipient_name,
        t.recipient_state = s.recipient_state,
        t.recipient_country = s.recipient_country,
        t.recipient_city = s.recipient_city
    WHEN NOT MATCHED THEN INSERT (
        recipient_uei, recipient_name, recipient_state, recipient_country, recipient_city
    ) VALUES (
        s.recipient_uei, s.recipient_name, s.recipient_state, s.recipient_country, s.recipient_city
    );

    SELECT @max_ts = MAX(_silver_load_ts) FROM dbo.vw_silver_unioned;

```

```
IF @max_ts IS NOT NULL
    UPDATE dbo.gold_watermark SET last_watermark = @max_ts WHERE table_name = 'dim_recipient';
END
GO
```

5. Incremental Load Procedure — Fact Table

T-SQL's MERGE does not support Spark SQL's null-safe equality operator (<=>). Each key column comparison is expanded below to: (t.col = s.col OR (t.col IS NULL AND s.col IS NULL)) — this is necessary because several of these key columns (e.g. program_activity_reporting_key) can legitimately be NULL, and standard = comparison in SQL never matches two NULLs.

agency_key, program_activity_key, and recipient_key are coalesced to -1 (the 'Unknown' member from Section 1.6) whenever the LEFT JOIN finds no match -- e.g. a genuinely NULL program_activity_code, confirmed as real source behavior, not a join defect. fiscal_year and period_number are derived from submission_period at load time.

```
CREATE PROCEDURE dbo.usp_LoadFactAward
AS
BEGIN
    DECLARE @watermark DATETIME2 = (SELECT last_watermark FROM dbo.gold_watermark WHERE table_name = 'fact_award');
    DECLARE @max_ts DATETIME2;

    MERGE INTO dbo.fact_award AS t
    USING (
        SELECT
            COALESCE(a.agency_key, -1) AS agency_key,
            ac.account_key,
            COALESCE(pa.program_activity_key, -1) AS program_activity_key,
            COALESCE(r.recipient_key, -1) AS recipient_key,
            s.award_unique_key, s.parent_award_id_piid, s.federal_account_symbol,
            s.program_activity_code, s.program_activity_name, s.object_class_code,
            s.direct_or_reimbursable_funding_source, s.disaster_emergency_fund_code,
            s.submission_period, s.program_activity_reporting_key, s.AwardCategory,
            s.transaction_obligated_amount, s.gross_outlay_amount_FYB_to_period_end,
            s.last_modified_date, s.agency_code,
            CONCAT('FY', SUBSTRING(s.submission_period, 3, 4)) AS fiscal_year,
            CAST(RIGHT(s.submission_period, 2) AS TINYINT) AS period_number
        FROM dbo.vw_silver_unioned s
        LEFT JOIN dbo.dim_agency a ON s.agency_code = a.agency_code
        LEFT JOIN dbo.dim_account ac ON s.federal_account_symbol = ac.federal_account_symbol
        LEFT JOIN dbo.dim_program_activity pa
            ON s.program_activity_code = pa.program_activity_code
            AND s.program_activity_name = pa.program_activity_name
        LEFT JOIN dbo.dim_recipient r ON s.recipient_uei = r.recipient_uei
        WHERE s._silver_load_ts > @watermark
    ) AS src
    ON (t.award_unique_key = src.award_unique_key
        OR (t.award_unique_key IS NULL AND src.award_unique_key IS NULL))
    AND (t.parent_award_id_piid = src.parent_award_id_piid
        OR (t.parent_award_id_piid IS NULL AND src.parent_award_id_piid IS NULL))
    AND (t.federal_account_symbol = src.federal_account_symbol
        OR (t.federal_account_symbol IS NULL AND src.federal_account_symbol IS NULL))
    AND (t.program_activity_code = src.program_activity_code
        OR (t.program_activity_code IS NULL AND src.program_activity_code IS NULL))
    AND (t.program_activity_name = src.program_activity_name
        OR (t.program_activity_name IS NULL AND src.program_activity_name IS NULL))
    AND (t.object_class_code = src.object_class_code
        OR (t.object_class_code IS NULL AND src.object_class_code IS NULL))
    AND (t.direct_or_reimbursable_funding_source = src.direct_or_reimbursable_funding_source
        OR (t.direct_or_reimbursable_funding_source IS NULL AND src.direct_or_reimbursable_funding_source IS NULL))
    AND (t.disaster_emergency_fund_code = src.disaster_emergency_fund_code
        OR (t.disaster_emergency_fund_code IS NULL AND src.disaster_emergency_fund_code IS NULL))
    AND (t.submission_period = src.submission_period
        OR (t.submission_period IS NULL AND src.submission_period IS NULL))
    AND (t.program_activity_reporting_key = src.program_activity_reporting_key
```



```

        OR (t.program_activity_reporting_key IS NULL AND src.program_activity_reporting_key IS NULL))
AND (t.AwardCategory = src.AwardCategory
     OR (t.AwardCategory IS NULL AND src.AwardCategory IS NULL))
WHEN MATCHED AND src.last_modified_date > t.last_modified_date THEN UPDATE SET
    t.agency_key = src.agency_key,
    t.account_key = src.account_key,
    t.program_activity_key = src.program_activity_key,
    t.recipient_key = src.recipient_key,
    t.transaction_obligated_amount = src.transaction_obligated_amount,
    t.gross_outlay_amount_FYB_to_period_end = src.gross_outlay_amount_FYB_to_period_end,
    t.last_modified_date = src.last_modified_date
WHEN NOT MATCHED THEN INSERT (
    agency_key, account_key, program_activity_key, recipient_key,
    award_unique_key, parent_award_id_piid, federal_account_symbol,
    program_activity_code, program_activity_name, object_class_code,
    direct_or_reimbursable_funding_source, disaster_emergency_fund_code,
    submission_period, program_activity_reporting_key, AwardCategory,
    transaction_obligated_amount, gross_outlay_amount_FYB_to_period_end,
    last_modified_date, agency_code, fiscal_year, period_number
) VALUES (
    src.agency_key, src.account_key, src.program_activity_key, src.recipient_key,
    src.award_unique_key, src.parent_award_id_piid, src.federal_account_symbol,
    src.program_activity_code, src.program_activity_name, src.object_class_code,
    src.direct_or_reimbursable_funding_source, src.disaster_emergency_fund_code,
    src.submission_period, src.program_activity_reporting_key, src.AwardCategory,
    src.transaction_obligated_amount, src.gross_outlay_amount_FYB_to_period_end,
    src.last_modified_date, src.agency_code, src.fiscal_year, src.period_number
);

SELECT @max_ts = MAX(_silver_load_ts) FROM dbo.vw_silver_unioned;
IF @max_ts IS NOT NULL
    UPDATE dbo.gold_watermark SET last_watermark = @max_ts WHERE table_name = 'fact_award';
END
GO

```

6. Orchestration

```
CREATE PROCEDURE dbo.usp_LoadGoldFull
AS
BEGIN
    EXEC dbo.usp_LoadDimAgency;
    EXEC dbo.usp_LoadDimAccount;
    EXEC dbo.usp_LoadDimProgramActivity;
    EXEC dbo.usp_LoadDimRecipient;
    EXEC dbo.usp_LoadFactAward;
END
GO
```

Call `usp_LoadGoldFull` from a Fabric pipeline's Stored Procedure activity, scheduled the same way as the Bronze/Silver notebooks (e.g. nightly).

7. Design Notes and Lessons Learned

These notes summarize the key debugging findings from building this pipeline, worth keeping for future reference or for explaining design decisions in a portfolio review.

7.1 Why the fact merge key has 10 columns

The natural expectation was that `award_unique_key` + `federal_account_symbol` + `program_activity_code` + `object_class_code` + `submission_period` uniquely identifies a row. Investigation showed this is false: 17 real, distinct transaction rows were found sharing that exact combination, with different `program_activity_name` values and different `transaction_obligated_amount` values. The fix was to widen the key to 10 columns (adding `parent_award_id_piid`, `program_activity_name`, `direct_or_reimbursable_funding_source`, `disaster_emergency_fund_code`, and `program_activity_reporting_key`), which was verified to be collision-free (`COUNT(*) = COUNT(DISTINCT ...)` returned 0 duplicate groups on both the assistance and contracts tables).

7.2 Why measure columns must never be part of a merge key

Including `transaction_obligated_amount` or `gross_outlay_amount` in the join/match key would break legitimate updates: if a source system later corrects an amount, the corrected row would have a different key value than the original, causing MERGE to insert it as a new row (via `WHEN NOT MATCHED`) instead of updating the existing one — resulting in duplicate rows and inflated totals. Amount comparison belongs only inside the `WHEN MATCHED ... AND` condition (comparing `last_modified_date`), never inside the `ON` clause.

7.3 Why deduplication (not aggregation) is used for exact-duplicate rows

An earlier version of this pipeline used `dropDuplicates()` on a too-narrow key, which silently collapsed genuinely distinct transaction rows into one, corrupting `SUM(transaction_obligated_amount)` (confirmed: Bronze totalled \$30,862,858,239.91 vs Silver's \$30,879,891,919.68 — an increase despite fewer rows, proving real data was being discarded rather than true duplicates). After widening the key to the verified 10-column combination, deduplication on that key is safe because it no longer collides with real, distinct rows.

7.4 'Unknown' dimension members and Direct Lake

A genuinely NULL business key (e.g. `program_activity_code`, confirmed real on ~358,600 rows via direct inspection of the raw CSV) produces a NULL foreign key on a LEFT JOIN. In Power BI's Direct Lake mode, a NULL foreign key can cause that fact row to silently disappear from any visual sliced by the corresponding dimension, rather than appearing as a visible '(Blank)' category the way Import mode typically handles it. Each dimension reserves surrogate key -1 for an explicit 'Unknown' member, and the fact load coalesces unmatched keys to -1, so the missing data remains visible and auditable in reports instead of vanishing.

7.5 Derived columns computed upstream, not in the semantic model

Direct Lake does not support DAX calculated columns. `fiscal_year` (formatted 'FY2025', matching the federal fiscal year convention and USAspending's own terminology, since the fiscal year runs October-September and does not align with the calendar year) and `period_number` (a numeric sort key, since `submission_period` like 'FY2025P10' sorts incorrectly as text) are therefore computed once, at load time, in the layer that writes `fact_award` -- not as columns added later in the reporting layer.

7.6 CSV parsing settings required

Source CSV files contain free-text description fields with embedded commas, quotes, and line breaks (e.g. prime_award_base_transaction_description). Reading these without explicit quote/escape/multiLine options caused column values to shift across rows. Required Spark CSV read options: quote="", escape="", multiLine=true.